Authentication

The platform has two separate authentication systems: **CRM Auth** (staff) and **Portal Auth** (external customers).

CRM Authentication (Staff)

Flow

```
POST /auth/login
  → validate email + bcrypt password
  → return { accessToken (24h JWT), refreshToken (7d JWT), user }

Client stores tokens → sends Authorization: Bearer <accessToken> on every request

POST /auth/refresh (with refreshToken in body)
  → issue new accessToken (no re-login required)

POST /auth/logout
  → clears refreshToken from DB
```

JWT Payload

```
{
  "sub": "user-cuid",
  "email": "user@example.com",
  "role": "ADMIN",
  "organizationId": "org-cuid",
  "iat": 1234567890,
```

```
  "exp": 1234654290
}
```

Guards

- `JwtAuthGuard` — most protected routes. Validates the Bearer token.
- `RolesGuard` — checks `user.role` against required roles (`@Roles('ADMIN')` decorator).

Password Management

- Passwords hashed with **bcryptjs** (10 salt rounds)
- `mustChangePassword: Boolean` flag — user is redirected to password change on login
- Admin can force reset: `POST /users/:id/reset-password`
- Self-service: `POST /auth/change-password`

User Statuses

Status	Behavior
`ACTIVE`	Normal login
`INACTIVE`	Cannot login
`SUSPENDED`	Login blocked, `suspendedAt` recorded
`LOCKED`	Login blocked after failed attempts, `lockedAt` recorded
`PENDING`	Account created, awaiting activation

Portal Authentication (External Customers)

Portal users are stored in a separate `PortalUser` table and have no access to CRM routes.

Flow

```
POST /portal/auth/login
  → validate email + bcrypt password for PortalUser
  → return { accessToken, user }
```

```
GET /portal/auth/password-policy
  → returns org's PortalSettings (min length, require uppercase, etc.)

POST /portal/auth/forgot-password
  → stores resetToken in DB, sends email

POST /portal/auth/reset-password { token, newPassword }
  → validates token expiry, resets password
```

Portal Guards

- `PortalAuthGuard` — validates portal JWT for `/portal/*` routes
- `PortalCrmAdminGuard` — requires `isPortalAdmin: true` for `/portal/padmin/*`

Account Activation

Portal users can be created by admins in two ways:

1. `POST /portal/admin/users` — admin creates an account with an initial password
2. `POST /portal/admin/records/:recordId/create-portal-user` — create portal user linked to a CRM record

On first login, if `isFirstLogin: true`, the portal redirects to password setup.

API Keys

For programmatic access, users can create API keys:

- Keys are stored as `bcrypt(key)` in `ApiKey.keyHash`; the plain key is shown once on creation
- Send as `Authorization: ApiKey <key>` header
- Keys can have expiry dates and are revocable

Token Storage (Frontend)

Tokens are stored in `localStorage` via the Zustand `auth.store.ts`. On app load, the store checks token expiry and calls `/auth/refresh` if needed.

Registration

`POST /auth/register` creates both a `User` and an `Organization` in one transaction. The first user in an org automatically gets `role: SUPER_ADMIN`.

Subsequent users are created by admins via `POST /users` — self-registration is not supported after the initial setup.